Report of CA2

AI-Genetic algorithms and Pentago game

Mahdi Zandevakili

Fitness Evaluation: At first let's explain what RMSE is. RMSE is the square root of the average of the squared differences between the predicted and actual values. Or in mathematical language: $\sqrt{(1/n * \sum_{i=1}^{n}(Ypre, i - Ytrue, i)^2)}$

But why this method? Well there are three advantages by using RMSE:

- Sensitive to large errors
- Same units as the original function
- Math friendly and wide usage

So, the main reason that we use RMSE, is because of the balanced error magnitude, it helps the algorithm focus on removing and reducing large errors.

Consider using other methods like MAE and R-squared method and why we do not use them.

MAE (Mean Absolute Error): the average of the absolute differences between predicted and actual values. Mathematically: $1/n * \sum_{i=1}^{n} |Ypre, i - Ytrue, i|$

What MAE is good for? straightforward, less sensitive measure of errors.

R-squared: measures how much of the variance in the target variable is explained by your model.

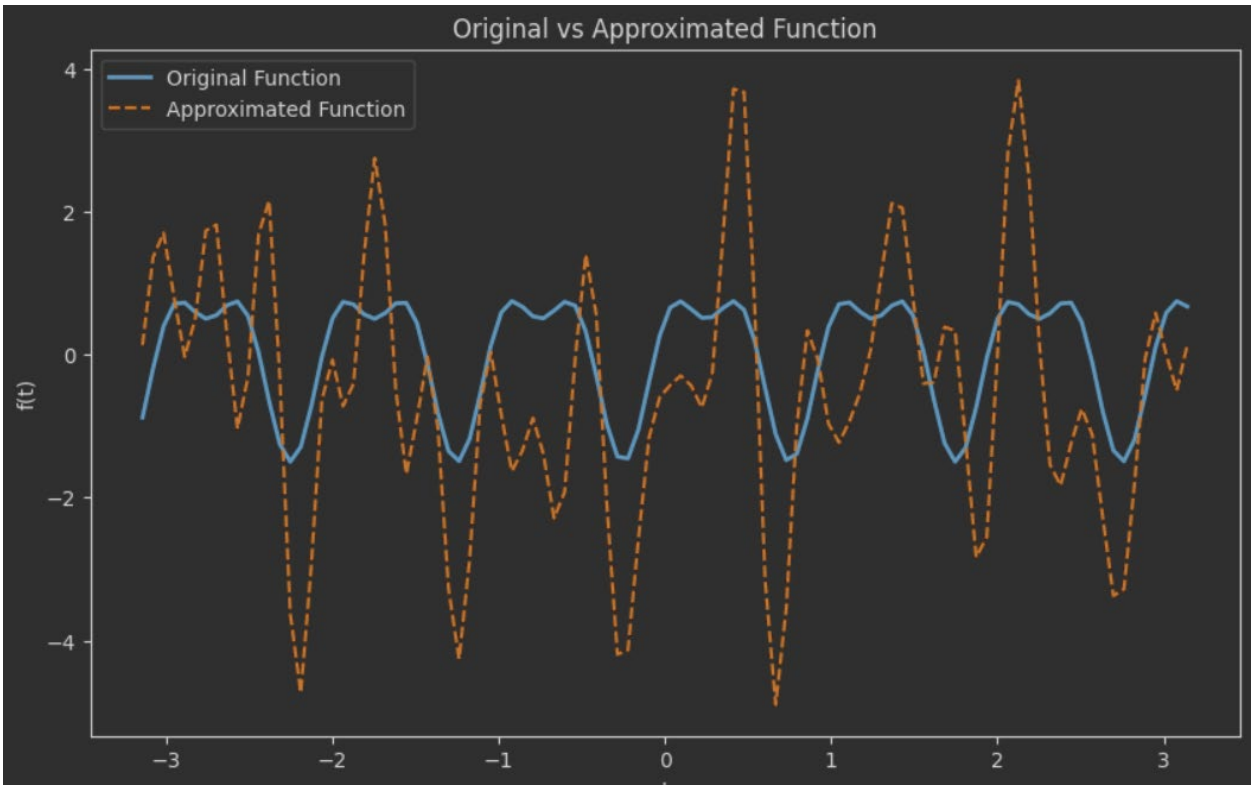
$R^2 = 1 - (\frac{\sum(Ytrue-Ypre)^2}{\sum(Ytrue-Y')^2})$ Y' is the actual values. It's great for comparing models, as higher R^2 indicates better performance.

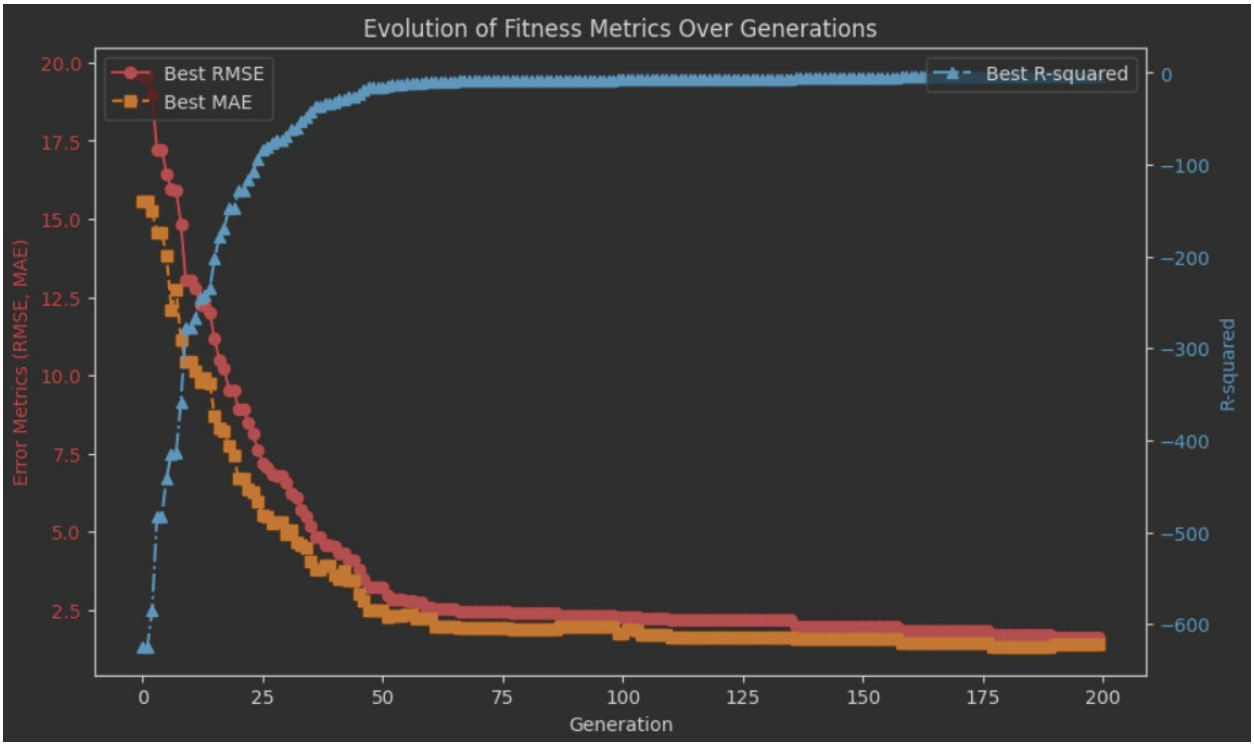
Below you 'll see the results of comparing these three methods for evaluation:

The test was ran for parameters below:

numCoeffs = 41
populationSize = 100
generations = 200
mutationRate = 0.15
functionRange = (-np.pi, np.pi)
sampleCount = 100
A = 10

and here are the figures:


Original vs Approximated Function
Original Function
Approximated Function
f(t)
4
2
0
−2
−4
−3
−2
−1
0
1
2
3


Evolution of Fitness Metrics Over Generations
Best RMSE
Best MAE
Best R-squared
Error Metrics (RMSE, MAE)
R-squared
Generation
20.0
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0
−100
−200
−300
−400
−500
−600
0
25
50
75
100
125
150
175
200

پاسخ سوالات بخش اول:

1. **هر کروموزوم به‌صورت یک بردار ۴۱ بعدی تعریف شده، که شامل ضرایب سری فوریه هم هست. اگر ضرایب به صورت پیوسته باشند، آنگاه تعداد حالات ممکن بینهایت است.**
2. **برای همگرا شدن مسئله دو ایده: اولین راهکار استفاده از Adaptive mutation rate هست؛ به این صورت که از نرخ جهش بالا در ابتدای فرایند برای ایجاد تنوع بهره می بریم، سپس در مراحل بعد، برای تثبیت پاسخ ها، نرخ را کم می کنیم.**
   ایده دوم: انتقال بخشی از بهترین کروموزوم‌ها (مثلاً ۱۰٪–۵ از جمعیت، عددی معقول) بدون تغییر به نسل بعدی، این روش تضمین می‌کند که اطلاعات خوب حفظ شده و از افت کیفیت به دلیل عملیات تصادفی جلوگیری شود.
3. **استراتژی اول:** Tournament Selection **به‌طور تصادفی چند کروموزوم انتخاب می‌شوند و بهترین شان به عنوان والد انتخاب می‌شود.**
   استراتژی دوم: Roulette Wheel Selection **در این روش، هر کروموزوم با احتمال متناسب با مقدار سازگاری** (fitness) **انتخاب می‌شود، سازگاری بیشتر، شانس انتخاب بالاتر.**
4. **برای جلوگیری از همگرایی زودرس، یکی از راه حل ها، افزایش نرخ جهش هست. به این صورت که کروموزوم ها تغییراتی جزئی ندارند و تنوع ژنتیکی حفظ می شود.**
   روش بعدی، شناسایی کروموزم های مشابه هست. بعد از شناسایی امتیازات آنان با نمونه های مشابه تقسیم می شود که این موضوع رقابت بین کروموزوم‌ها را کاهش داده و به حفظ تنوع هم کمک می‌کند.
5. **بالاتر نیز توضیح هایی داده شده! (اختلاف بین مقدار پیش‌بینی شده و مقدار واقعی برای هر نمونه محاسبه و به توان دو می‌رسدو در نهایت میانگین این مقادیر محاسبه می‌شود.)**
   در حالی که R-squared **به ما اطلاعات خوبی از تطابق کلی مدل با داده‌ها می‌دهد، ممکن هست حساسیت لازم نسبت به خطاهای مطلق را مثل دیگر متد ها نداشته باشد؛ به همین دلیل از آن (معمولا) به عنوان یک مکمل برای ارزیابی عملکرد ها استفاده میشه. (در مدل‌های غیرخطی یا وقتی تغییرات جزئی در مدل داریم، به طور قابل توجهی خطای مطلق رو کاهش می‌ده. و این بده اگر تنها متدمون باشه!)**

Second part – Pentago game

1. **افزایش عمق جستجو در الگوریتم Minimax به معنای بررسی حرکات بیشتر و دقیق‌تر است. این موضوع باعث می‌شود:**

   اول،زمان محاسبه و تعداد نودهای بازدید شده افزایش یابد.

   دوم،تصمیمات بهینه‌تری اتخاذ شود که شانس برد را افزایش می‌دهد.

   یعنی رابطه‌ای مستقیم بین عمق، زمان اجرا، تعداد گره‌های دیده شده و موفقیت بازی وجود دارد.

2. **بله، اگر بتوان فرزندان هر نود را بر اساس تخمین اولیه از ارزش (heuristic) مرتب کرد، بهترین حرکات ابتدا بررسی می‌شوند و این کار هرس آلفا-بتا را موثرتر می‌کند؛ زیرا شاخه‌هایی که با اطمینان می‌توان آن‌ها را رد کرد، سریع‌تر از درخت جستجو حذف می‌شوند.**

3. **به تعداد حرکات ممکن (شاخه‌های درخت جستجو) در هر نود گفته می‌شود. با پیشرفت بازی تعداد حرکات ممکن کاهش پیدا می‌کند؛ یعنی branching factor کمتر می‌شود. این کاهش باعث می‌شود که فضای جستجو کوچکتر شده و الگوریتم سریع‌تر به تصمیم نهایی برسد.**

4. **هرس آلفا-بتا با حذف شاخه‌هایی از درخت که بدون شک بدتر از بهترین حالت‌های موجود هستند (با استفاده از مقایسه مقادیر آلفا و بتا) عمل می‌کند.**

   این کار باعث می‌شود بدون از دست دادن دقت نهایی، تعداد نودهای مورد بررسی به طرز چشمگیری کاهش یافته و زمان محاسبه بهبود یابد.

5. **وقتی حریف به صورت شانسی عمل می‌کند، استفاده از الگوریتم Minimax که بر پایه‌ی فرض بهینه بودن حرکات حریف است، می‌تواند کارایی چندانی نداشته باشد؛ زیرا در این حالت حریف هیچ استراتژی مشخصی ندارد.**

   به همین دلیل، الگوریتم‌هایی که بر اساس شبیه‌سازی‌های تصادفی عمل می‌کنند، می‌توانند جایگزین مناسبی برای Minimax باشند.